

Session 07 (Binding, Polymorphism & Interfaces)

Part 1: Static Binding (Early Binding) — Method Hiding with new

Q1. Create a class Shape with two auto-properties Width and Height (of type double), a constructor that initializes them, and a method Area() that returns Width * Height. Override ToString() to return "(Width = ..., Height = ...)".

Q2. Create a class Cube that inherits from Shape:

- Add a new property Depth initialized through the constructor (use : base(...)).
- Hide the Area() method using the new keyword so it returns base.Area() * Depth.
- Add a Print() method that prints all three dimensions.

Q3. In Main, create:

- Shape shape = new Shape(2, 3); → call Area() — what is printed?
- Cube cube = new Cube(2, 3, 4); → call Area() — what is printed?
- Shape shapeRef = new Cube(2, 3, 4); → call Area() — what is printed and why? Explain the difference between this and the previous call (which version of Area() gets executed and why this is called early/static binding).

Q4. Declare object obj = new Cube(1, 2, 3); and call obj.ToString(). Which ToString() runs and why? What concept does this demonstrate (late/dynamic binding)? Why did ToString() behave polymorphically while Area() did not?

Part 2: Dynamic Binding (Late Binding) — virtual / override

Q5. Create a base class Person with properties ID, Name, Age, and two methods:

- Greet() (non-virtual) that prints "I am a Person."
- Display() marked virtual that prints the person's basic data.

Q6. Create two derived classes:

- Doctor : Person with property Specialty — hide Greet() with new and override Display() to include the specialty.
- Engineer : Person with properties Field and YearsOfExperience — hide Greet() with new and override Display() to include all data.

Q7. Write a method ProcessPerson(Person person) that calls person.Greet() and person.Display(). Then:

- Create a Doctor object (using object initializer syntax) stored in a Person reference and pass it to ProcessPerson.
- Create an Engineer object the same way and pass it too.
- Predict the output before running. Which method calls are resolved at compile time and which at runtime? Why does Greet() print the base version while Display() prints the derived version?

Q8. What happens if you remove virtual from Display() in Person? What compiler warning/error appears in the derived classes, and what does it mean?

Part 3: Interfaces

Q9. (Pre-interface problem) Create a class Car with methods MoveForward() and MoveBackward() that just print messages. Now imagine you also need MoveUp()/MoveDown(). What is the problem with forcing every vehicle class to implement all four methods? (This is what interfaces solve.)

Q10. Create two interfaces:

- IMoveable → MoveForward(), MoveBackward()
- IFlyable → MoveUp(), MoveDown()

Q11. Implement the interfaces in three classes:

- Car : IMoveable — prints messages about moving on the ground.
- Ship : IMoveable — prints messages about moving on the sea.
- Airplane : IMoveable, IFlyable — prints messages about moving in the air.

Q12. In Main:

- Create each class with its concrete type and call its methods.
- Then declare IMoveable carRef = new Car(); and IMoveable planeRef = new Airplane(); and call MoveForward()/MoveBackward() through the interface references.
- Question: Can you call MoveUp() on planeRef? Why or why not? What reference type would you need?

Q13. Create an interface IVehicle that inherits from both IMoveable and IFlyable without adding new members. Then create a class Vehicle : IVehicle that implements all four methods as virtual. What is the benefit of an interface inheriting other interfaces?

Q14. (Challenge) Can a class implement IMoveable explicitly? Rewrite Ship.MoveForward() using explicit interface implementation (void IMoveable.MoveForward()), then try calling it on a Ship object directly. What happens, and how must you call it instead?

Part 4: Comprehension / Theory

Q15. Fill in the comparison table from your own experiments:

Feature	Static Binding (new)	Dynamic Binding (override)
Keyword in base	?	?
Keyword in derived	?	?
Resolved at	?	?
Behavior via base reference	?	?

Q16. In your own words: why does C# require virtual on the base method before override is allowed, but new works on any method?

Would you like me to also generate a model answer/solution project for these questions, or a graded rubric for evaluating students' submissions?